

**VIETNAM INTERNATIONAL UNIVERSITY
HO CHI MINH UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**Microprocessors-Microcontrollers (CO3010)
LAB Report**

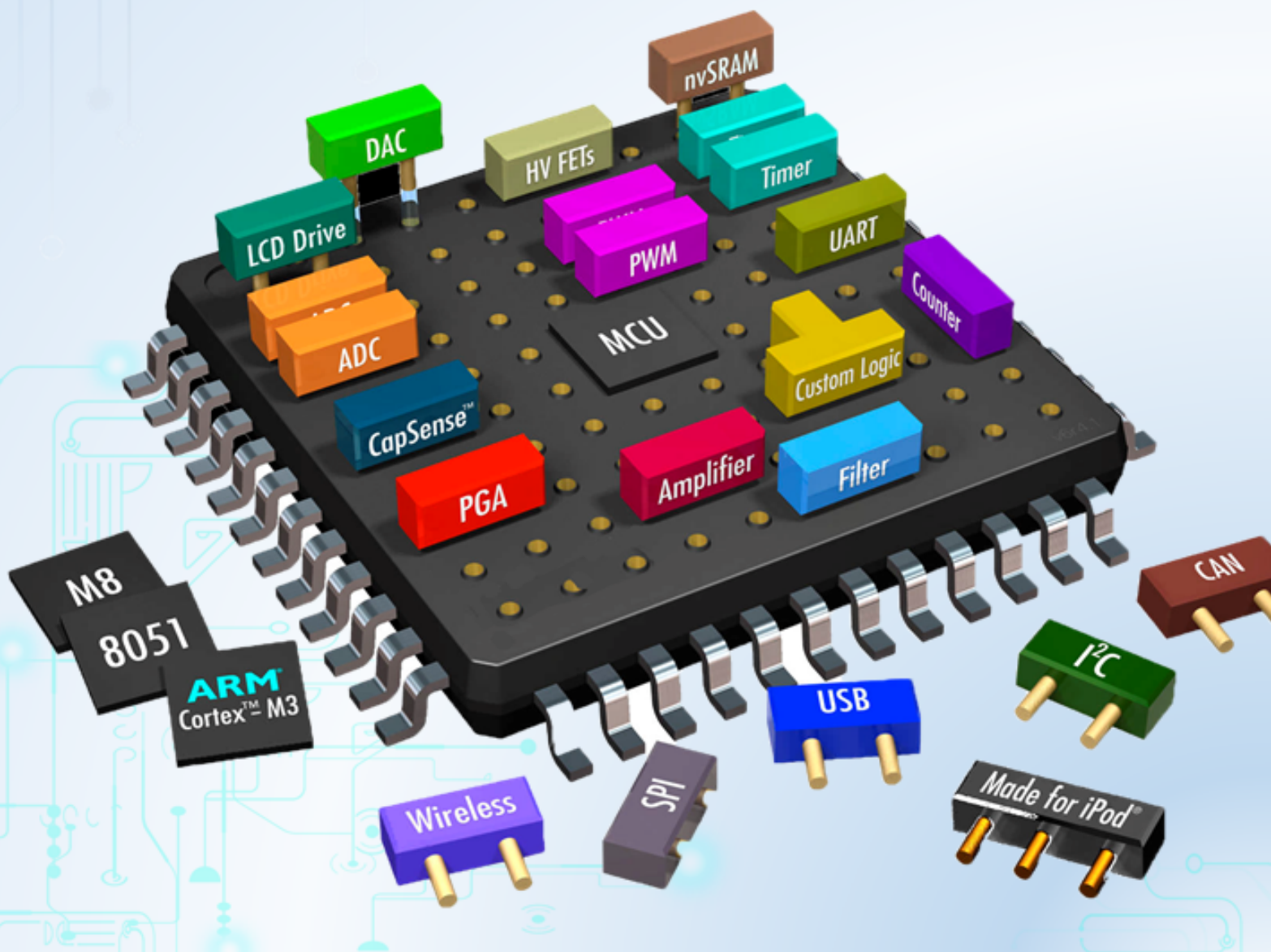
Student Name	Student ID
Tran Chi An	2352080

Contents

Chapter 1. Flow and Error Control in Communication	5
1 LAB 5 Report	6
1.1 Header Includes	6
1.2 Global Definitions and Variables	6
1.3 UART Interrupt Handler	7
1.4 FSM 1: Command Parser	8
1.5 FSM 2: UART Communication Logic	9
1.6 Main Function Implementation	10
1.7 Github link	11

CHAPTER 1

Flow and Error Control in Communication



1 LAB 5 Report

1.1 Header Includes

The following standard C libraries were included to support string manipulation and formatting.

```
1 /* USER CODE BEGIN Includes */
2 #include "stdio.h"
3 #include "string.h"
4 /* USER CODE END Includes */
```

Program 1.1: Included Libraries

Explanation:

- `stdio.h`: This library is included to use the `sprintf` function. This is necessary for formatting the ADC (Analog-to-Digital Converter) value into the required string format (e.g., `!ADC=1234#`).
- `string.h`: This library is included for two functions:
 - `strncmp`: Used in the command parser to compare the incoming buffer against the protocol commands (`!RST#` and `!OK#`).
 - `strlen`: Used to calculate the exact length of the formatted string before transmitting it via UART.

1.2 Global Definitions and Variables

These constants and variables are defined to manage the state of the FSMs (Finite State Machines), communication buffers, and protocol-specific values.

```
1 /* USER CODE BEGIN PD */
2 #define MAX_BUFFER_SIZE 30
3 #define TIMEOUT_DURATION 3000
4 #define CMD_NONE 0
5 #define CMD_RST 1
6 #define CMD_OK 2
7 #define STATE_IDLE 0
8 #define STATE_WAIT_OK 1
9 /* USER CODE END PD */
```

Program 1.2: Global Definitions

```
1 /* USER CODE BEGIN PV */
2 uint8_t temp = 0;
3 uint8_t buffer[MAX_BUFFER_SIZE];
4 uint8_t index_buffer = 0;
5 uint8_t buffer_flag = 0;
6
7 uint8_t g_command_flag = CMD_NONE;
```

```

8 uint8_t g_communication_state = STATE_IDLE;
9 uint32_t g_timer_start = 0;
10 uint32_t g_last_adc_value = 0;
11 char tx_buffer[50];
12 /* USER CODE END PV */

```

Program 1.3: Global Variables

Explanation:

- **Definitions (PD):** These constants make the code more readable.
 - MAX_BUFFER_SIZE: Defines the maximum size of the reception buffer.
 - TIMEOUT_DURATION: Sets the timeout period to 3000ms (3 seconds) as required by the lab.
 - CMD_...: Define the possible commands that the parser FSM can identify.
 - STATE_...: Define the states for the main communication FSM.
- **Variables (PV):**
 - temp, buffer, index_buffer, buffer_flag: These variables implement the UART reception buffer. temp holds the single byte from the interrupt, buffer stores the accumulating command, index_buffer tracks the position, and buffer_flag is a flag set by the interrupt to signal the main loop.
 - g_command_flag: A global flag used by command_parser_fsm to notify uart_communication_fsm of a newly received command.
 - g_communication_state: Tracks the current state of the main protocol (either STATE_IDLE or STATE_WAIT_OK).
 - g_timer_start: Stores the timestamp (from HAL_GetTick()) when the ADC data was sent, used to check for the 3-second timeout.
 - g_last_adc_value: Stores the ADC value at the time of the !RST# command. This is crucial for re-transmitting the *same* value if a timeout occurs.
 - tx_buffer: A character array used by sprintf to build the outgoing message.

1.3 UART Interrupt Handler

This function is the Interrupt Service Routine (ISR) for UART. It is automatically triggered by the HAL (Hardware Abstraction Layer) every time a single byte is successfully received.

```

1 /* USER CODE BEGIN 0 */
2 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
3     if(huart->Instance == USART2){
4         if (index_buffer < MAX_BUFFER_SIZE) {
5             buffer[index_buffer++] = temp;
6         }
7         buffer_flag = 1;
8         HAL_UART_Receive_IT(&huart2, &temp, 1);
9     }

```

```
10 }
```

Program 1.4: UART Receive Complete Callback

Explanation: This function implements the logic from Program 1.4. Its sole purpose is to handle incoming data quickly and efficiently:

1. It checks if the interrupt is from USART2.
2. It appends the received byte (stored in `temp`) to the buffer.
3. It sets `buffer_flag = 1`. This flag signals the main `while(1)` loop that new data is ready to be parsed.
4. It re-enables the UART interrupt using `HAL_UART_Receive_IT` to listen for the next byte.

1.4 FSM 1: Command Parser

This function, `command_parser_fsm`, is responsible for interpreting the data collected in the buffer.

```
1 void command_parser_fsm() {
2     if (index_buffer > 0 && buffer[index_buffer - 1] == '#') {
3         if (index_buffer == 5 && strncmp((char*)buffer, "!RST#", 5) == 0) {
4             g_command_flag = CMD_RST;
5         }
6         else if (index_buffer == 4 && strncmp((char*)buffer, "!OK#", 4) == 0) {
7             g_command_flag = CMD_OK;
8         }
9         index_buffer = 0;
10    }
11    if (index_buffer >= MAX_BUFFER_SIZE) {
12        index_buffer = 0;
13    }
14 }
```

Program 1.5: Command Parser FSM

Explanation: This FSM is called from the main loop whenever `buffer_flag` is set.

1. It first checks if the most recently received character is `#`, which signifies the end of a command.
2. If it is, it checks the buffer content. It uses `strncmp` and the known lengths to see if the buffer matches `!RST#` or `!OK#`.
3. If a match is found, it sets the `g_command_flag` accordingly.
4. In all cases (match or no match), `index_buffer` is reset to 0, effectively clearing the buffer to be ready for the next command.

5. An additional check prevents a buffer overflow by resetting the index if MAX_BUFFER_SIZE is reached without a #.

1.5 FSM 2: UART Communication Logic

This function, `uart_communication_fsm`, is the main "brain" of the protocol. It manages the communication state, handles sending data, and processes timeouts.

```
1 void uart_communication_fsm() {
2     switch (g_communication_state) {
3         // === Waiting for command state ===
4         case STATE_IDLE:
5             // If FSM 1 -> CMD_RST
6             if (g_command_flag == CMD_RST) {
7                 // 1. Read ADC
8                 g_last_adc_value = HAL_ADC_GetValue(&hadc1)
9             };
10            // 2. Send !ADC=xxxx#
11            sprintf(tx_buffer, "!ADC=%lu#",
12                g_last_adc_value);
13            HAL_UART_Transmit(&huart2, (uint8_t*)
14                tx_buffer, strlen(tx_buffer), 200);
15            // 3. Start timer and switch state
16            g_timer_start = HAL_GetTick();
17            g_communication_state = STATE_WAIT_OK;
18            // 4. Delete flag
19            g_command_flag = CMD_NONE;
20        }
21        break;
22        // === Waiting OK ===
23        case STATE_WAIT_OK:
24            // 1. Check if OK ?
25            if (g_command_flag == CMD_OK) {
26                g_communication_state = STATE_IDLE;
27                g_command_flag = CMD_NONE;
28            }
29            // 2. Check TIMEOUT (3s)
30            if (HAL_GetTick() - g_timer_start >
31                TIMEOUT_DURATION) {
32                sprintf(tx_buffer, "!ADC=%lu#",
33                    g_last_adc_value);
34                HAL_UART_Transmit(&huart2, (uint8_t*)
35                    tx_buffer, strlen(tx_buffer), 200);
36                g_timer_start = HAL_GetTick();
37            }
38        }
39        break;
40    }
41 }
```

Program 1.6: UART Communication FSM

Explanation: This FSM is managed by a switch statement on the `g_communication_state` variable.

- **STATE_IDLE:** This is the default state. The FSM waits for the `command_parser_fsm` to set `g_command_flag` to `CMD_RST`. When this happens, it:
 1. Reads the current ADC value using `HAL_ADC_GetValue`.
 2. Saves this value to `g_last_adc_value`.
 3. Formats the output string `!ADC=... #`.
 4. Transmits the string via UART.
 5. Starts the 3-second timeout timer by recording `HAL_GetTick()`.
 6. Transitions the state to `STATE_WAIT_OK`.
- **STATE_WAIT_OK:** In this state, the FSM waits for an acknowledgment.
 1. **If `CMD_OK` is received:** The protocol is complete. It resets the state to `STATE_IDLE`.
 2. **If no `CMD_OK` is received:** It continuously checks if 3 seconds have passed. If the timeout occurs, it re-transmits the *previously saved* value (`g_last_adc_value`) and restarts the 3-second timer.

1.6 Main Function Implementation

The main function is modified to initialize the ADC and UART systems and to run the two FSMs in the main loop.

```
1 /* USER CODE BEGIN 2 */
2   HAL_ADC_Start(&hadc1);
3   HAL_UART_Receive_IT(&huart2, &temp, 1);
4 /* USER CODE END 2 */
```

Program 1.7: Initialization in `main()`

```
1 /* Infinite loop */
2 /* USER CODE BEGIN WHILE */
3 while (1)
4 {
5   uart_communiation_fsm();
6   if (buffer_flag == 1) {
7     command_parser_fsm();
8     buffer_flag = 0;
9   }
10  /* USER CODE END WHILE */
```

Program 1.8: Main Loop `while(1)`

Explanation:

- **Initialization (USER CODE BEGIN 2):**

- HAL_ADC_Start(&hadc1): Since the ADC is configured in ContinuousConvMode, this function starts the ADC, which will continuously sample the input. HAL_ADC_GetValue can then be called at any time to get the latest conversion.
- HAL_UART_Receive_IT(&huart2, &temp, 1): This function is called once at the beginning to "prime" the UART interrupt system, enabling it to receive the very first byte.
- **Main Loop** (USER CODE BEGIN WHILE): This loop follows the structure of Program 1.6.
 - uart_communication_fsm(): This function is called **continuously** in every loop iteration. This is critical for ensuring the timeout logic (which relies on HAL_GetTick()) is checked as frequently as possible.
 - if (buffer_flag == 1): This block is executed **only** when the UART ISR has set the flag. It calls command_parser_fsm() to process the new data and then immediately clears the buffer_flag to acknowledge that the buffer has been processed.

1.7 Github link

https://github.com/Tran-Chi-An-1104/STM32_LAB5